

Steps to Host a 3-Tier Architecture Website on AWS

Architecture: Internet → Internet-facing Load Balancer → Web Server → Internal Load Balancer → App Server → Amazon RDS

Step 1: Create the VPC

Create a VPC with 2 public subnets, 4 private subnets, 2 NAT Gateways, and 1 Internet Gateway. Use two Availability Zones. Public subnets route Internet traffic through the Internet Gateway; private subnets use NAT Gateways for outbound Internet access.

Step 2: Create Security Groups

Create five security groups: Internet-LB-SG, Web-Server-SG, Internal-LB-SG, App-Server-SG, and Database-SG. Allow traffic only from the required previous tier.

```
Internet-LB-SG:  TCP 80,443  ← 0.0.0.0/0
Web-Server-SG:   TCP 80      ← Internet-LB-SG
Internal-LB-SG:  TCP 5000     ← Web-Server-SG
App-Server-SG:   TCP 5000     ← Internal-LB-SG
Database-SG:     TCP 3306     ← App-Server-SG
```

Step 3: Create an RDS Subnet Group

Create an RDS subnet group using private subnets in two different Availability Zones, for example Private Subnet 3 in AZ-A and Private Subnet 4 in AZ-B. Make sure the RDS instance is not publicly accessible.

Step 4: Create the Web Server

Create an Amazon Linux EC2 instance in a public subnet with a public IP address. Attach Web-Server-SG and name the instance Web-Server. Connect using PuTTY or SSH.

Step 5: Install Apache HTTP Server

```
sudo yum install httpd -y
sudo systemctl start httpd
sudo systemctl enable httpd
sudo systemctl status httpd
```

Step 6: Transfer Frontend Files to the Web Server

Use WinSCP to transfer the frontend files from Windows to /home/ec2-user/. Then copy them into Apache's document root.

```
sudo cp -r /home/ec2-user/* /var/www/html/
ls -la /var/www/html/
sudo systemctl restart httpd
```

Step 7: Create the App Server

Create an Amazon Linux EC2 instance in a private subnet. Disable public IP assignment and attach App-Server-SG. Name it App-Server. The App Server should not be directly accessible from the Internet.

Step 8: Prepare the SSH Key on the Web Server

Copy the EC2 private key to the Web Server and restrict its permissions.

```
vi key.pem
chmod 400 key.pem
ls -l key.pem
```

Step 9: Connect to the App Server from the Web Server

Because the App Server has no public IP, connect through the Web Server. The correct SSH option is `-i`.

```
ssh -i key.pem ec2-user@PRIVATE_IP_OF_APP_SERVER
```

```
# Example
```

```
ssh -i key.pem ec2-user@10.0.137.35
```

Step 10: Transfer Backend Files to the App Server

Use `scp` from the Web Server to transfer the backend project to the private App Server.

```
scp -i key.pem -r library_management ec2-user@10.0.137.35:/home/ec2-user/
```

```
# Then connect
```

```
ssh -i key.pem ec2-user@10.0.137.35
```

Step 11: Create the .env File

Go to the backend directory and create `.env`. Use the RDS endpoint DNS name, not the RDS ARN.

```
cd ~/library_management
```

```
vi .env
```

```
DB_HOST=your-rds-endpoint
```

```
DB_PORT=3306
```

```
DB_NAME=library_management
```

```
DB_USER=admin
```

```
DB_PASSWORD=your-password
```

Step 12: Install Python and the MySQL Client

```
sudo yum update -y
```

```
sudo yum install python3 -y
```

```
python3 --version
```

```
# MySQL/MariaDB client (Amazon Linux version dependent)
```

```
sudo dnf install mariadb105 -y
```

```
mysql --version
```

Step 13: Install Python Dependencies

```
cd ~/library_management
```

```
pip3 install -r requirements.txt
```

```
# Alternative
```

```
python3 -m pip install -r requirements.txt
```

Step 14: Seed the Database

If the project uses a Python seed script, run it with Python. If it uses an SQL file, import it using the MySQL client.

```
python3 seed.py
```

```
# Or
```

```
mysql -h YOUR_RDS_ENDPOINT -P 3306 -u admin -p library_management < seed.sql
```

```
# Verify
```

```
mysql -h YOUR_RDS_ENDPOINT -P 3306 -u admin -p
```

```
USE library_management;
```

```
SHOW TABLES;
```

Step 15: Start the Flask Application

Start Flask in the background and inspect the process and logs. The application must listen on `0.0.0.0:5000` so the Internal Load Balancer can reach it.

```
nohup python3 app.py > output.log 2>&1 &
```

```
ps -ef | grep app.py
```

```
cat output.log
```

```
tail -f output.log

# app.py should use:
app.run(host="0.0.0.0", port=5000)
```

Step 16: Test the App Server

Test the local Flask application, health endpoint, and login API.

```
curl http://localhost:5000
curl http://localhost:5000/health
```

```
curl -X POST http://localhost:5000/api/login -H "Content-Type: application/json" -d '{"email":"borrower@library.local",
```

Step 17: Configure Apache Reverse Proxy on the Web Server

On the Web Server, configure Apache to forward /api/ requests to the Internal Load Balancer.

```
sudo nano /etc/httpd/conf.d/library_proxy.conf

<VirtualHost *:80>
    ProxyPreserveHost On
    ProxyPass      /api/ http://INTERNAL-LB-DNS-NAME:5000/api/
    ProxyPassReverse /api/ http://INTERNAL-LB-DNS-NAME:5000/api/
</VirtualHost>

sudo httpd -t
sudo systemctl restart httpd
sudo systemctl status httpd
```

Step 18: Create the Target Group for the Internal Load Balancer

Create an instance target group using HTTP on port 5000. Register the App Server. Configure the health check as HTTP on the traffic port with path /health. The Flask application should return HTTP 200 from GET /health.

```
@app.route("/health")
def health():
    return {"status": "healthy"}, 200
```

Step 19: Create the Internal Load Balancer

Create an internal Application Load Balancer in private subnets across two Availability Zones. Configure an HTTP listener on port 5000 and forward traffic to the App-Server target group.

Step 20: Test the Internal Load Balancer

From the Web Server, verify that the Internal Load Balancer can reach the App Server.

```
curl http://INTERNAL-LB-DNS-NAME:5000/health

curl -X POST http://INTERNAL-LB-DNS-NAME:5000/api/login -H "Content-Type: application/json" -d '{"email":"borrower@lib
```

Step 21: Create the Target Group for the Internet-Facing Load Balancer

Create an instance target group using HTTP on port 80. Register the Web Server. Configure the health check path as /login.html and ensure it returns HTTP 200.

Step 22: Create the Internet-Facing Load Balancer

Create an internet-facing Application Load Balancer in the two public subnets. Create an HTTP listener on port 80 and an HTTPS listener on port 443 when SSL is configured. Forward traffic to the Web-Server target group. Do not configure the Internet-facing ALB to directly forward traffic to the App Server.

Step 23: Configure Route 53 and SSL

Create a Route 53 hosted zone for your domain. Request an SSL certificate from AWS Certificate Manager (ACM), complete DNS validation, and use the certificate on the ALB's HTTPS listener.

Step 24: Create a Route 53 Record

Create an A record with Alias enabled and point it to the Internet-facing Application Load Balancer. Your domain can then resolve to the ALB.

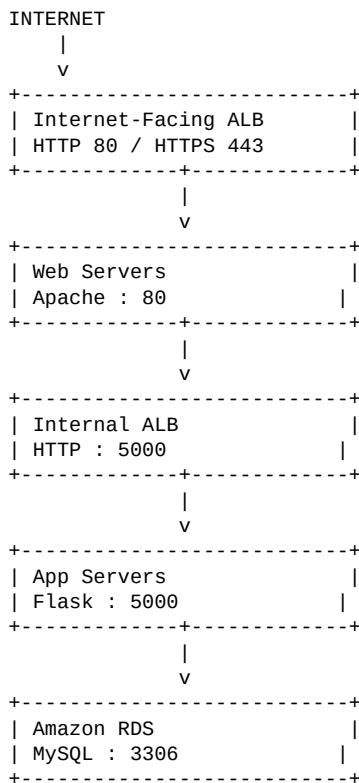
Step 25: Create AMIs and Launch Templates

Create a separate AMI and Launch Template for each tier. The Web Server Launch Template should use Web-Server-AMI and Web-Server-SG. The App Server Launch Template should use App-Server-AMI and App-Server-SG, with the App Server remaining in private subnets.

Step 26: Create Auto Scaling Groups

Create a Web Server Auto Scaling Group using the Web Server Launch Template, place it in the public subnets, and attach it to the Web-Server target group. Create a separate App Server Auto Scaling Group using the App Server Launch Template, place it in private subnets, and attach it to the App-Server target group. Example capacity for each tier: Minimum 2, Desired 2, Maximum 4.

Final 3-Tier Traffic Flow



Important Security Principle

Each tier should communicate only with the required next tier: Internet → Internet-facing ALB → Web Server → Internal ALB → App Server → RDS. The App Server and RDS should not be publicly accessible.